



안양대학교 일반대학원 · 미세먼지관리전공

GIT & GITHUB

대학원 논문을 위한 디지털 연구 도구 특강
Session 1 — 버전 관리의 기초부터 실전까지

담당 정광용 교수

대상 미세먼지관리전공 대학원생

시간 2시간 (Zoom)

오늘의 목표

이 세션이 끝나면 할 수 있는 것



할 수 있게 되는 것

- ▶ Git 저장소를 처음부터 만들기
- ▶ 파일 변경을 스냅샷으로 저장
- ▶ GitHub에 연구 코드 업로드
- ▶ 과거 버전으로 되돌리기
- ▶ 담당 교수님과 코드 공유



실습 결과물

- ▶ pm25-thesis 저장소 생성
- ▶ data / analysis / figures / docs 구조
- ▶ 첫 커밋 완료
- ▶ GitHub에 push 완료
- ▶ README.md 작성

💡 이 수업의 철학: 이론보다 실습. 오늘 실제로 pm25-thesis 저장소를 만들고, 첫 커밋까지 완료합니다.

문 제 인 식

논문 쓰면서 이런 경험 있으신가요?



파일명 지옥

- ▶ thesis_final.docx
- ▶ thesis_final_v2.docx
- ▶ thesis_진짜최종.docx
- ▶ thesis_최종수정본.docx
- ▶ thesis_제출용_진짜.docx



코드 손실

- ▶ PM2.5 분석 잘 되던 코드
- ▶ 수정했다가 완전히 망함
- ▶ 이전 버전으로 못 돌아감
- ▶ 어느 버전이 맞는지 불명
- ▶ 처음부터 다시 작성...



협업 혼돈

- ▶ 교수님과 카카오톡으로 파일 공유
- ▶ 누가 최신 버전인지 모름
- ▶ 동시에 수정 → 충돌
- ▶ 수정 이력 추적 불가
- ▶ "어디서 바뀐 거죠?" 질문 반복

 Git은 이 모든 문제를 해결합니다 — 연구자의 타임머신이자 협업 도구

개념 이해

Git vs GitHub — 뭐가 다른가?



Git (로컬 도구)

- ▶ 내 컴퓨터에 설치하는 소프트웨어
- ▶ 파일 변경 이력을 스냅샷으로 저장
- ▶ 언제든지 과거 버전으로 복원 가능
- ▶ 인터넷 없이도 동작
- ▶ 1991년 리누스 토르발스가 개발

버전관리

로컬

무료



GitHub (클라우드 플랫폼)

- ▶ Git 저장소를 인터넷에 백업
- ▶ 어디서든 접근 가능
- ▶ 교수님과 코드 공유 & 리뷰
- ▶ 논문 코드 공개 저장소로 활용
- ▶ Microsoft가 운영 (2018년 인수)

클라우드

협업

공개/비공개



비유: Git = 내 컴퓨터의 타임머신 기능 / GitHub = 그 타임머신 데이터를 저장하는 클라우드

핵심 개념

버전 관리란? — 스냅샷의 연속



커밋 = 스냅샷

파일 전체를 복사하는 것이 아니라, 변경된 내용만 기록합니다. 각 스냅샷에는 누가, 언제, 무엇을 바꿨는지 기록됩니다.



이력 = 타임라인

모든 커밋이 시간 순서로 연결됩니다. 1년 전 분석 코드로도 돌아갈 수 있고, 특정 시점의 상태를 그대로 재현할 수 있습니다.

```
# git log --oneline 으로 보는 커밋 이력
a7f3c21 AI 예측 모델: Random Forest RMSE 12.3 달성
b92e1d4 PM2.5 계절별 분석: 봄철 고농도 원인 규명
c541f8a 데이터 정제: AirKorea 결측값(-999) 제거
d103a77 초기 설정: 수도권 PM2.5 연구 저장소 구축

# ↑ 아무 커밋이든 git checkout [해시] 로 그 시점으로 이동 가능
```

실습 준비

Git 설치 & 초기 설정

1

Git 설치

git-scm.com → Download → Windows 사용자는 Git for Windows 설치 → Git Bash 포함 여부 확인

2

GitHub 계정 생성

github.com → Sign up → 계정 생성 → 사용자명은 영문+숫자로 설정

3

터미널에서 신원 등록 (아래 명령어 실행)

커밋에 내 이름이 표시되도록 설정. GitHub 계정과 동일한 이름 사용 권장

이름 설정

```
git config --global user.name "홍길동"
```

기본 브랜치 이름 설정 (최신 표준)

```
git config --global init.defaultBranch main
```

Windows 권장: 줄바꿈 자동 변환

```
git config --global core.autocrlf true
```

설정 확인

```
git config --list
```

copy

 **Windows 체크포인트:** 시작 메뉴에서 **Git Bash**를 열어 `git --version` 실행 → 첫 push 때 뜨는 GitHub 로그인 창에서 브라우저 인증을 완료합니다. 비밀번호를 직접 입력하지 않고 Git Credential Manager가 인증을 저장합니다.

핵심 개념

Git의 3단계 공간 — 반드시 이해해야 할 것



왜 Staging Area가 있나?

여러 파일을 수정했지만 일부만 커밋하고 싶을 때 사용합니다. 예: 분석 코드 완성 + 실험적 코드 → 분석 코드만 커밋

실수로 `git add` 했다면?

`git reset HEAD 파일명`
스테이징 취소. 파일 내용은 그대로.

실습 1

pm25-thesis 저장소 만들기

① 바탕화면 또는 원하는 위치에 연구 폴더 생성

mkdir pm25-thesis && **cd** pm25-thesis

② Git 저장소 초기화 (.git 숨김 폴더가 생성됨)

git init

Initialized empty Git repository in ~/pm25-thesis/.git/

③ 권장 폴더 구조 생성

mkdir data analysis figures docs**mkdir** data/raw data/processed

④ 기본 파일 생성

touch README.md .gitignore**touch** analysis/pm25_analysis.py**touch** docs/thesis_outline.md

⑤ 현재 상태 확인

git status

Untracked files: README.md, .gitignore, analysis/, ...

copy

중요 설정

.gitignore — 추적하지 않을 파일 설정

⚠ 반드시 설정해야 합니다! 측정 원시 데이터(.csv, .nc), 대용량 파일, 개인정보가 담긴 파일이 GitHub에 올라가면 심각한 문제가 됩니다.

.gitignore 파일에 작성할 내용 (미세먼지 연구 전용)

copy

대용량 원시 데이터

data/raw/

***.csv**

***.nc** # NetCDF (기상/대기 모델 출력)

***.h5** # HDF5 (에어로졸 측정 데이터)

***.mat** # MATLAB 데이터

Python 임시 파일

__pycache__/

***.pyc**

.ipynb_checkpoints/

***.egg-info/**

.env # API 키 등 환경변수 (절대 올리지 말 것!)

OS / 에디터 파일

.DS_Store # macOS

Thumbs.db # Windows

.vscode/

***.swp**

README.md 작성 — 연구 저장소 소개

README.md 템플릿 (이 내용을 복사해서 수정하세요)

copy

수도권 PM2.5 농도 분포 특성 연구

안양대학교 일반대학원 환경공학과 미세먼지관리전공
대학원 학위논문 연구 코드 저장소

연구 개요

- 연구 주제: [본인 연구 주제 입력]
- 담당 교수: 정광용 교수님
- 연구 기간: 2024.03 ~ 2025.02
- 사용 데이터: AirKorea 측정망, 기상청 관측 자료

폴더 구조

```
pm25-thesis/  
├── data/raw/ # 원시 데이터 (gitignore됨)  
├── data/processed/ # 정제된 데이터  
├── analysis/ # Python 분석 스크립트  
├── figures/ # 논문용 그래프  
└── docs/ # 논문 초고
```

실행 방법

```
pip install -r requirements.txt  
python analysis/pm25_analysis.py
```

첫 번째 커밋 만들기

① 스테이징 (커밋할 파일 선택)

```
git add README.md .gitignore
```

② 커밋 - 스냅샷 저장 (메시지는 '무엇을 했는가'로)

```
git commit -m "초기 설정: 수도권 PM2.5 연구 저장소 구축"
```

[main (root-commit) a3f9c21] 초기 설정: 수도권 PM2.5 연구 저장소 구축

2 files changed, 28 insertions(+)

③ analysis 폴더도 추가

```
git add analysis/
```

```
git commit -m "분석 구조: pm25_analysis.py 초안 추가"
```

④ 커밋 이력 확인

```
git log --oneline
```

b92e1d4 분석 구조: pm25_analysis.py 초안 추가

a3f9c21 초기 설정: 수도권 PM2.5 연구 저장소 구축

copy

👉 좋은 커밋 메시지 원칙: 50자 이내, 현재형 동사, 무엇을 왜 했는지

"PM2.5 데이터 정제: AirKorea -999 결측값 제거 및 NaN 처리"

GitHub에 저장소 만들고 연동하기

1

GitHub에서 새 Repository 생성

github.com → 우측 상단 + → New repository → 이름: pm25-thesis → Private 선택 → Create repository

2

로컬 저장소와 GitHub 연결

GitHub 페이지에서 안내하는 명령어 복사 후 터미널에 붙여넣기

```
# 원격 저장소 주소 등록
git remote add origin https://github.com/내아이디/pm25-thesis.git

# 업로드 (처음 push 시 -u 옵션으로 추적 브랜치 설정)
git push -u origin main
Enumerating objects: 6, done.
Branch 'main' set up to track remote branch 'main' from 'origin'.

# 이후부터는 간단하게
git push
```

copy

치트 시트

매일 쓰는 Git 명령어 7가지

git status

변경된 파일 목록 확인. 가장 자주 쓰는 명령어. 무엇이든 하기 전에 먼저 실행

git add .

변경된 모든 파일 스테이징. 특정 파일만: **git add analysis/pm25.py**

git commit -m "..."

스냅샷 저장. 분석 단계마다, 기능 완성 때마다 의미있는 메시지 작성

git push

로컬 커밋을 GitHub에 업로드. 하루 작업 끝날 때 반드시 실행

git pull

GitHub의 최신 변경사항을 로컬로 다운로드. 작업 시작 전 실행 습관

git log --oneline

커밋 이력 간략히 보기. **--graph** 추가하면 브랜치 시각화

git checkout [커밋ID] -- [파일명]

특정 파일을 과거 버전으로 복원. 실수로 코드 망쳤을 때 구원자!

실용 명령어

변경 내용 확인 — diff와 show

아직 스테이징 전인 변경사항 보기

git diff

- pm25_threshold = 25 # 국내 기준

+ pm25_threshold = 35 # WHO 기준으로 변경

스테이징된 변경사항 보기

git diff --staged

특정 커밋의 내용 보기

git show a3f9c21

두 커밋 사이의 차이 비교

git diff a3f9c21..b92e1d4

특정 파일의 변경 이력만 보기

git log --oneline -- analysis/pm25_analysis.py

누가 어떤 줄을 수정했는지 (blame)

git blame analysis/pm25_analysis.py

copy

복구 전략

실수했을 때 — 되돌리기 3가지

1

Working Directory 되돌리기

`git checkout -- 파일명`

수정한 파일을 마지막 커밋 상태로 복원. 커밋 전
에만 가능. 복원 후 되돌릴 수 없음!

2

Staging 취소

`git reset HEAD 파일명`

`git add` 한 것을 취소. 파일 내용은 그대로. 다시
수정 후 `add` 가능

3

커밋 메시지 수정

`git commit --amend`

직전 커밋 메시지 수정. `push` 전에만 사용! `push`
후에는 절대 사용 금지

😬 무서워하지 마세요! Git에서 커밋된 내용은 거의 지워지지 않습니다. 실수를 되돌릴 수 있기 때문에 Git을 사용하는 겁니다.

정리할 곳

이제 원본 리포지토리로 이동합니다.

공식문 (2020년)

워크플로우

미세먼지 연구자의 일일 Git 루틴



연구 시작 전 — git pull

어제 집에서 수정한 내용, 또는 교수님이 검토한 내용을 받아오기. 항상 최신 상태에서 시작



연구 작업 중 — 파일 수정 & git status 반복

Python 분석, AERMOD 파라미터 조정, 그래프 생성 등 실제 연구. 수시로 git status로 변경사항 확인



작업 단위 완료 시 — git add + git commit

의미있는 단위(기능 완성, 분석 완료)마다 커밋. "하루에 1번"이 아니라 "의미있는 단위마다"



작업 종료 — git push

하루 연구 GitHub에 업로드. 컴퓨터가 망가져도 코드는 안전. 교수님이 GitHub에서 확인 가능

전공 연계

미세먼지관리전공 수업에서 Git 활용



대기확산모델링 이론

AERMOD, CALPUFF 입력 파라미터 파일(.inp)과 실행 스크립트를 Git으로 관리. 어떤 파라미터 조합이 최적인지 커밋 이력으로 추적. 실험 실패도 기록으로 남김



파이썬 기반 환경 통계분석

수업 실습 코드 → Git으로 저장 → 논문 분석으로 발전. pandas, matplotlib, scipy 스크립트를 체계적으로 관리하며 재현 가능한 연구 구현



AI기반 미세먼지 예측 및 관리

ML 모델 코드, 하이퍼파라미터 실험 이력, 예측 결과 비교. Git 없이는 어떤 모델이 어떤 설정에서 RMSE가 낮았는지 추적 불가



SMART 측정 및 분석 실습

센서 데이터 처리 파이프라인 코드를 Git으로 관리. 측정 방법 변경 이력을 커밋으로 기록 → 논문 재현성 확보

requirements.txt — 패키지 목록 관리

💡 **왜 필요한가?** 교수님이나 동료가 내 코드를 실행하려면 같은 라이브러리가 필요합니다. requirements.txt가 없으면 "ModuleNotFoundError"의 연속입니다.

현재 설치된 패키지를 자동으로 기록

```
pip freeze > requirements.txt
```

requirements.txt 내용 예시 (미세먼지 연구)

pandas=2.1.0 # 데이터 처리

numpy=1.26.0 # 수치 계산

matplotlib=3.8.0 # 그래프

seaborn=0.13.0 # 통계 시각화

scipy=1.11.3 # 통계 분석

scikit-learn=1.3.1 # 머신러닝

folium=0.15.0 # 지도 시각화

netCDF4=1.6.4 # 기상 데이터 (.nc)

다른 컴퓨터에서 한 번에 설치

```
pip install -r requirements.txt
```

copy

GITHUB 활용

GitHub 웹에서 할 수 있는 것들



코드 온라인 뷰어

별도 소프트웨어 없이 Python, CSV 파일을 브라우저에서 바로 확인. 교수님이 어디서든 코드 열람 가능



Insights — 기여도 시각화

언제 얼마나 코드를 작성했는지 그래프로 확인. 연구 활동량을 객관적으로 보여줄 수 있음



코드 검색

저장소 내에서 특정 함수명, 변수명 검색. 대규모 분석 코드에서 특정 부분 빠르게 찾기



모바일 앱

이동 중에도 저장소 확인, Issue 등록, 코드 리뷰 가능. 교수님과 빠른 소통



GitHub Education: GitHub Student Developer Pack 신청 → GitHub Pro 무료 + 다양한 개발 도구 무료 사용

문 제 해 결

자주 만나는 오류와 해결법

① push 거부됨 - 원격에 더 새로운 커밋 있음

error: failed to push some refs

git pull --rebase origin main # 먼저 pull 후 다시 push

② 인증 오류 - GitHub 로그인 문제

remote: Support for password authentication was removed

GitHub → Settings → Developer settings → Personal access tokens → 토큰 생성

비밀번호 대신 토큰 사용

③ 실수로 민감한 파일 커밋함

git rm --cached data/raw/secret.csv # 추적 해제

git commit -m "데이터 파일 추적 해제"

.gitignore에 추가 후 push (GitHub에 이미 올라간 건 직접 삭제 필요)

④ 저장소 주소 확인/변경

git remote -v

git remote set-url origin https://github.com/새주소.git

copy

세션 요약

Session 1 핵심 정리



오늘 배운 개념

- ▶ Git vs GitHub 차이
- ▶ 3단계 공간 (Working/Staging/Repo)
- ▶ 커밋 = 스냅샷
- ▶ .gitignore 설정
- ▶ 일일 Git 루틴



핵심 명령어

- ▶ git init / status
- ▶ git add / commit
- ▶ git push / pull
- ▶ git log / diff
- ▶ git checkout (복원)



다음 세션 예고

- ▶ 브랜치(Branch) 전략
- ▶ 교수님과 PR 리뷰
- ▶ 충돌(Conflict) 해결
- ▶ GitHub Issues 활용
- ▶ Fork & Clone

✅ 오늘 목표: pm25-thesis 저장소 GitHub에 생성 + 첫 커밋 3개 이상 완료!

과제

Session 1 과제



다음 세션 전까지 완료

1. GitHub 계정 생성 후 **[이름]_pm25thesis** 저장소 생성 (Private)
예: honggildong_pm25thesis
2. 로컬에 폴더 구조 생성: **data/raw / data/processed / analysis / figures / docs**
.gitignore에 *.csv, *.nc, data/raw/ 추가 필수
3. README.md 작성: 본인 연구 주제, 담당 교수, 사용 데이터, 폴더 설명 포함
아직 연구 주제 없으면 임시 주제로 작성 OK
4. 의미있는 커밋 5개 이상 완료 후 GitHub push
각 커밋마다 다른 내용을 담은 메시지 작성
5. Windows 사용자는 **Git Bash 실행 화면 + git config --list 결과 확인**
core.autocrlf=true, user.name, init.defaultBranch=main 확인
6. requirements.txt 파일 생성 (현재 설치된 패키지 목록)
pip freeze > requirements.txt 실행
7. 🎁 **보너스:** analysis/pm25_analysis.py에 샘플 데이터 로드 코드 작성 + 커밋
파이썬 기반 환경 통계분석 수업 내용 연계 권장